

Fundamental Limitation in Explaining AI

A Quadrilemma among Environment Complexity, AI Performance, Interpretability, and Faithfulness

Atsushi Suzuki Jing Wang

The University of Hong Kong / University of Greenwich



香港大學

THE UNIVERSITY OF HONG KONG

KAWAII lab

Materials available here!

- arXiv: <https://arxiv.org/abs/2605.24727>



- The presentation slides:

https://ash-suzuki.github.io/material/xai_limitation_talk.pdf



SUZUKI, Atsushi

- 2011-2020: Bachelor (Engineering, 2015), Master (Information Science and Technology (IST), 2017), PhD (IST, 2020), The University of Tokyo
- 2020-2024: Assistant Professor (UK Lecturer) in UK (University of Greenwich, King's College London)
- 2025-: Assistant Professor (Math), **The University of Hong Kong.**

Research interests:

- Methods for AI (differential geometry, information theory, deep learning)
- Theoretical analysis of AIs (statistics, information theory, computability theory)

Outline

1. Motivation
2. Problem Formulation
3. Formulating the 4 conditions
4. Main Result
5. Conclusion
6. References

Success of Als

Success of AIs

- McKinsey estimates its annual economic value to be **\$2.6–\$4.4 trillion** [10].
- Goldman Sachs estimates it could boost **global GDP by about 7% (approximately \$7 trillion)** over the next decade [4].

Social demand in explaining AIs

Public institutions emphasize the importance of **explainability in AI**.

Social demand in explaining AIs

Public institutions emphasize the importance of **explainability in AI**.

Example (Action Plans by NIST (US) [11])

- “Address general risks associated with a lack of **explainability and transparency** in GAI systems” (GV-4.1-001)
- “Implement interpretability and explainability methods to evaluate GAI system decisions and verify alignment with intended purpose.” (MS-4.2-003)

Existing explanations are not completely “faithful”

We say that an explanation is **faithful** if we can recover the behavior of AI ¹.

¹There is no single established and formalized definition of “faithful,” to the best of our knowledge

Existing explanations are not completely “faithful”

We say that an explanation is **faithful** if we can recover the behavior of AI ¹.

The faithfulness must be a plausible property, because, if otherwise, it means that the explanation lacks some information about the AI's behavior.

However,

¹There is no single established and formalized definition of “faithful,” to the best of our knowledge

Existing explanations are not completely “faithful”

We say that an explanation is **faithful** if we can recover the behavior of AI ¹.

The faithfulness must be a plausible property, because, if otherwise, it means that the explanation lacks some information about the AI's behavior.

However, existing explanation methods do NOT pursue complete faithfulness.

¹There is no single established and formalized definition of “faithful,” to the best of our knowledge

Existing explanations are not completely “faithful”

We say that an explanation is **faithful** if we can recover the behavior of AI ¹.

The faithfulness must be a plausible property, because, if otherwise, it means that the explanation lacks some information about the AI’s behavior.

However, existing explanation methods do NOT pursue complete faithfulness.

- **Requiring scalarization of the output:** LIME [13], SHAP [9], integrated gradients [18], Grad-CAM [16] etc.
- **Generating a text:** LLM-based approaches, e.g., [5–7].
- **surrogate models:** e.g., LIME [13], Anchors [14], GLocalX [17], GLEAMS [19].

¹There is no single established and formalized definition of “faithful,” to the best of our knowledge

Non faithful explanation example: LIME

Example (LIME (Local Interpretable Model-agnostic Explanations) [13])

- Considers a scalar output and binarization of the original data points, i.e., intentionally **gives up the faithfulness** of the explanation.
- Explanation: local approximation in the binarized space.

Specifically, fix a binarization function $(\bullet)_{\text{bin}} : \mathcal{X} \rightarrow \{0, 1\}^d$ and recover function $(\bullet)_{\text{rec}} : \{0, 1\}^d \rightarrow \mathcal{X}$. The explanation of a function $f : \mathcal{X} \rightarrow \mathbb{R}$ of interest around x is given by $w_x \in \mathbb{R}^{d'}$ that minimizes

$$\sum_{z'_{\text{rec},x} \in \text{neighbor}(x_{\text{bin}})} \text{similarity}(x, z'_{\text{rec},x}) \left(f(z'_{\text{rec},x}) - w_x^\top z'_{\text{rec},x} \right)^2 \quad \text{s.t. } \|w_x\|_0 \leq K.$$

Starting question

Starting question

Why did they have to be not completely faithful to an AI?

Naïve question: **Can we not provide a faithful explanation** of AIs?

Starting question

Why did they have to be not completely faithful to an AI?

Naïve question: **Can we not provide a faithful explanation** of AIs?

Before we discuss this problem, we first need to exclude trivial and uninteresting cases.

Exclude trivial cases. 1. When AI's performance is poor

Example (A poor AI case)

Task: AI is asked to answer the user's question.

Exclude trivial cases. 1. When AI's performance is poor

Example (A poor AI case)

Task: AI is asked to answer the user's question.

- **User prompt 1:** Tell me the smallest perfect number?
- **AI's answer 1:** I am an AI.

Exclude trivial cases. 1. When AI's performance is poor

Example (A poor AI case)

Task: AI is asked to answer the user's question.

- **User prompt 1:** Tell me the smallest perfect number?
- **AI's answer 1:** I am an AI.
- **User prompt 2:** Where is the capital city of Japan?
- **AI's answer 2:** I am an AI.

Exclude trivial cases. 1. When AI's performance is poor

Example (A poor AI case)

Task: AI is asked to answer the user's question.

- **User prompt 1:** Tell me the smallest perfect number?
- **AI's answer 1:** I am an AI.
- **User prompt 2:** Where is the capital city of Japan?
- **AI's answer 2:** I am an AI.
- **Explanation:** "The AI just outputs I am an AI., i.e., $\text{AI}(x) = \text{I am an AI.}$ "

In this case, while the explanation is faithful, the AI's performance is poor.

Exclude trivial cases. 2. When the task/environment is simple.

Example (Trivial task case.)

Task: AI is asked to repeat the user prompt.

Exclude trivial cases. 2. When the task/environment is simple.

Example (Trivial task case.)

Task: AI is asked to repeat the user prompt.

- **User prompt 1:** Tell me the smallest perfect number.
- **AI's answer 1:** Tell me the smallest perfect number.

Exclude trivial cases. 2. When the task/environment is simple.

Example (Trivial task case.)

Task: AI is asked to repeat the user prompt.

- **User prompt 1:** Tell me the smallest perfect number.
- **AI's answer 1:** Tell me the smallest perfect number.
- **User prompt 2:** Where is the capital city of Japan?
- **AI's answer 2:** Where is the capital city of Japan?

Exclude trivial cases. 2. When the task/environment is simple.

Example (Trivial task case.)

Task: AI is asked to repeat the user prompt.

- **User prompt 1:** Tell me the smallest perfect number.
- **AI's answer 1:** Tell me the smallest perfect number.
- **User prompt 2:** Where is the capital city of Japan?
- **AI's answer 2:** Where is the capital city of Japan?
- **Explanation:** "The AI just repeats the user prompt, i.e., $AI(x) = y$ "

In this case, the AI's performance is perfect, and the explanation is faithful.

Exclude trivial cases. 3. When AI explanation is NOT interpretable.

Example (A non-interpretable explanation case)

Task: AI is asked to answer the user's question.

Exclude trivial cases. 3. When AI explanation is NOT interpretable.

Example (A non-interpretable explanation case)

Task: AI is asked to answer the user's question.

- **User prompt 1:** Tell me the smallest perfect number?
- **AI's answer 1:** It's 6.

Exclude trivial cases. 3. When AI explanation is NOT interpretable.

Example (A non-interpretable explanation case)

Task: AI is asked to answer the user's question.

- **User prompt 1:** Tell me the smallest perfect number?
- **AI's answer 1:** It's 6.
- **User prompt 2:** Where is the capital city of Japan?
- **AI's answer 2:** The de facto capital city of Japan is Tokyo.

Exclude trivial cases. 3. When AI explanation is NOT interpretable.

Example (A non-interpretable explanation case)

Task: AI is asked to answer the user's question.

- **User prompt 1:** Tell me the smallest perfect number?
- **AI's answer 1:** It's 6.
- **User prompt 2:** Where is the capital city of Japan?
- **AI's answer 2:** The de facto capital city of Japan is Tokyo.
- **Explanation:** (Just gives the source code and the checkpoint)

In this case, while the AI performs well and the explanation is faithful, the explanation is NOT interpretable.

Refined research question

Research question: Can there exist an AI and its explanation such that:

- The AI works in a complicated environment, e.g., natural language Q/A.
- The AI's performance is good.
- The explanation is completely faithful to the AI behavior.
- The explanation is interpretable.

Main answer

Theorem (Informal quadrilemma)

*An AI system and its explanation **CANNOT** simultaneously satisfy all of the following:*

- The AI's operation environment is complicated,*
- The AI's performance is good,*
- The explanation is completely faithful to the AI behavior,*
- The explanation is interpretable.*

Main answer

Theorem (Informal quadrilemma)

*An AI system and its explanation **CANNOT** simultaneously satisfy all of the following:*

- The AI's operation environment is complicated,*
- The AI's performance is good,*
- The explanation is completely faithful to the AI behavior,*
- The explanation is interpretable.*

But a natural question arises: how do we **mathematically** formulate the above notions?

Main answer

Theorem (Informal quadrilemma)

*An AI system and its explanation **CANNOT** simultaneously satisfy all of the following:*

- The AI's operation environment is complicated,*
- The AI's performance is good,*
- The explanation is completely faithful to the AI behavior,*
- The explanation is interpretable.*

But a natural question arises: how do we **mathematically** formulate the above notions?

Actually, our research's main contribution is its **formulation**, rather than proofs.

Difficulty in mathematically formulating the interpretability

We may mathematically formulate the conditions as below (rough idea):

- The AI's operation environments complexity: Statistics of the data distribution, function space, etc.

Difficulty in mathematically formulating the interpretability

We may mathematically formulate the conditions as below (rough idea):

- The AI's operation environments complexity: Statistics of the data distribution, function space, etc.
- The AI's performance: Evaluation criteria, e.g., perplexity

Difficulty in mathematically formulating the interpretability

We may mathematically formulate the conditions as below (rough idea):

- The AI's operation environments complexity: Statistics of the data distribution, function space, etc.
- The AI's performance: Evaluation criteria, e.g., perplexity
- The explanation's faithfulness to the AI behavior: e.g., Whether the explanation can be converted to the implementation

Difficulty in mathematically formulating the interpretability

We may mathematically formulate the conditions as below (rough idea):

- The AI's operation environments complexity: Statistics of the data distribution, function space, etc.
- The AI's performance: Evaluation criteria, e.g., perplexity
- The explanation's faithfulness to the AI behavior: e.g., Whether the explanation can be converted to the implementation

But...

Difficulty in mathematically formulating the interpretability

We may mathematically formulate the conditions as below (rough idea):

- The AI's operation environments complexity: Statistics of the data distribution, function space, etc.
- The AI's performance: Evaluation criteria, e.g., perplexity
- The explanation's faithfulness to the AI behavior: e.g., Whether the explanation can be converted to the implementation

But... **How** can we mathematically define the **interpretability** of an explanation?

Key trick to avoid the difficulty in formulating the interpretability

It's not easy to formulate the interpretability of an explanation [1, 8].

Key trick to avoid the difficulty in formulating the interpretability

It's not easy to formulate the interpretability of an explanation [1, 8].

Key trick: To negate a condition, it's sufficient to disprove its **necessary** condition.

Key trick to avoid the difficulty in formulating the interpretability

It's not easy to formulate the interpretability of an explanation [1, 8].

Key trick: To negate a condition, it's sufficient to disprove its **necessary** condition.

What condition is

- necessary for the interpretability, and
- easy to formulate mathematically?

Key trick to avoid the difficulty in formulating the interpretability

It's not easy to formulate the interpretability of an explanation [1, 8].

Key trick: To negate a condition, it's sufficient to disprove its **necessary** condition.

What condition is

- necessary for the interpretability, and
- easy to formulate mathematically?

Our idea: Use the **shortness** of the explanation as a necessary condition for the interpretability.

Why the shortness is necessary for the interpretability?

An explanation must be short to be interpretable since human capacity to read and remember text is limited [2] [3]

Why the shortness is necessary for the interpretability?

An explanation must be short to be interpretable since human capacity to read and remember text is limited [2] [3]

Example (Human memorizing capacity [3])

The world record for memorizing the decimal expansion of $\pi = 3.14159\dots$, is 70,000 digits [3], which is at most 233 thousand bits.

Why the shortness is necessary for the interpretability?

An explanation must be short to be interpretable since human capacity to read and remember text is limited [2] [3]

Example (Human memorizing capacity [3])

The world record for memorizing the decimal expansion of $\pi = 3.14159\dots$, is 70,000 digits [3], which is at most 233 thousand bits.

Example (Human reading capacity [2])

238 English words per minute for nonfiction text, with 4.6 characters per word, which corresponds to approximately 4B bits per year.

Why the shortness is necessary for the interpretability?

An explanation must be short to be interpretable since human capacity to read and remember text is limited [2] [3]

Example (Human memorizing capacity [3])

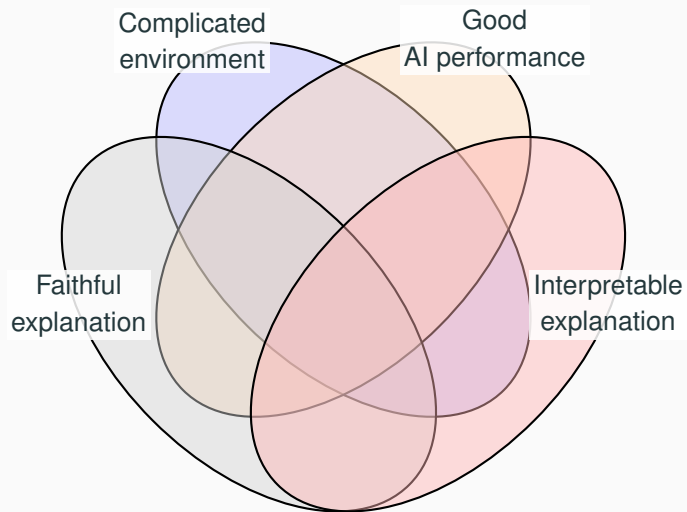
The world record for memorizing the decimal expansion of $\pi = 3.14159\dots$, is 70,000 digits [3], which is at most 233 thousand bits.

Example (Human reading capacity [2])

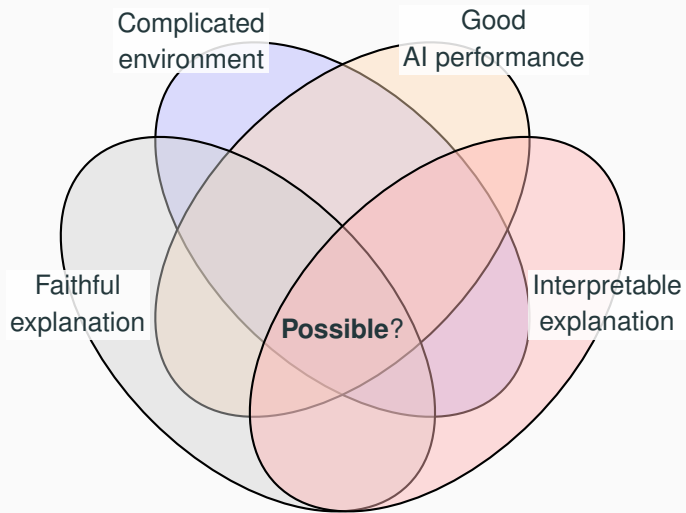
238 English words per minute for nonfiction text, with 4.6 characters per word, which corresponds to approximately 4B bits per year.

This is why the pair of the source code and checkpoint (28B bits even for a 7B model with the 4 bit quantization.) of the open model is NOT interpretable.

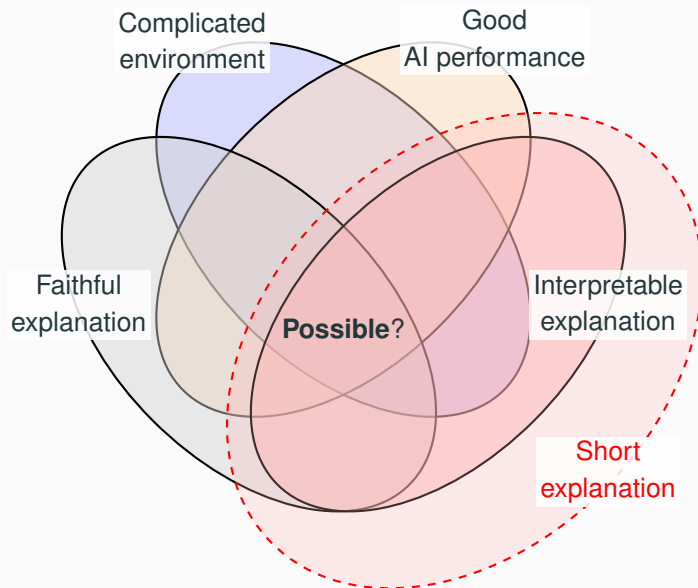
Conceptual picture



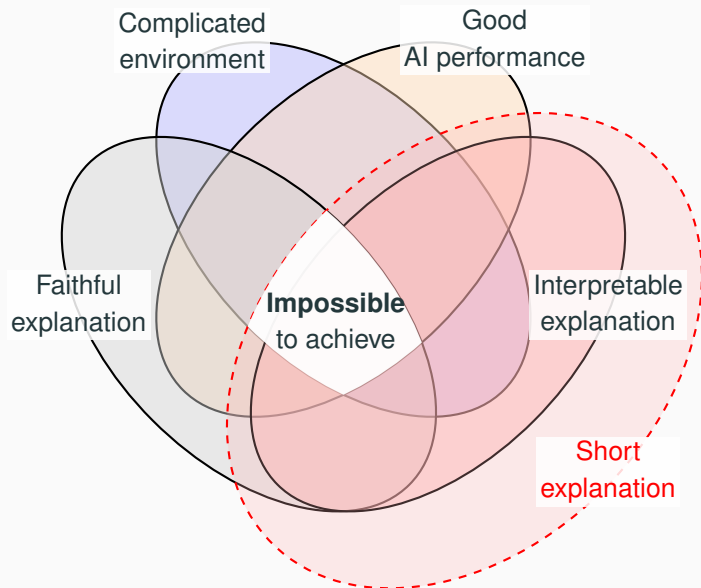
Conceptual picture



Conceptual picture



Conceptual picture



What we actually prove is...

Theorem (Informal quadrilemma (more formulatable))

*An AI system and its explanation **CANNOT** simultaneously satisfy all of the following:*

- *The AI's operation environment is complicated,*
- *The AI's performance is good,*
- *The explanation is completely faithful to the AI behavior,*
- *The explanation is **short**.*

The above statement is much easier to formulate than the original form.

Outline

1. Motivation

2. Problem Formulation

3. Formulating the 4 conditions

4. Main Result

5. Conclusion

6. References

Outline

1. Motivation

2. Problem Formulation

3. Formulating the 4 conditions

4. Main Result

5. Conclusion

6. References

What do we need to do?

- Formulate an AI itself.
- Formulate the goodness of AI performance.
- Formulate the faithfulness of the explanation.
- Formulate the shortness of the explanation.
- Formulate AI's operation environment

Preliminary: finite-length strings

Definition (Finite-length string)

Let Σ be the letter set that the computer can natively use. Typically $\Sigma = 0, 1$. A **finite-length string** is a finite-length sequence of letters, e.g., 001101011.

The set of all finite-length strings is denoted by $\Sigma^* := \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$.

Note that in a computer, everything needs to be represented as a finite-length string, and the set Σ^* is countably infinite.

Hence, it CANNOT represent the whole of an uncountably infinite set such as the real number set $\mathbb{R}$ or the function class $\{f : \Sigma^* \rightarrow \Sigma^*\}$.

Rough formulation of AI

We define an AI as an input-output stochastic computation process

Definition (A rough formulation of an AI)

A **stochastic input-output AI** is a system that receives

Rough formulation of AI

We define an AI as an input-output stochastic computation process

Definition (A rough formulation of an AI)

A **stochastic input-output AI** is a system that receives

- **Task Input** $x \in \Sigma^*$: User prompt, system prompt, user context, etc, and (often)

Rough formulation of AI

We define an AI as an input-output stochastic computation process

Definition (A rough formulation of an AI)

A **stochastic input-output AI** is a system that receives

- **Task Input** $x \in \Sigma^*$: User prompt, system prompt, user context, etc, and (often)
- **Random sequence** $u \in \Sigma^*$ generated by using a pseudo random number generator until it satisfies the termination condition $u \in \mathcal{T}_x$,

Rough formulation of AI

We define an AI as an input-output stochastic computation process

Definition (A rough formulation of an AI)

A **stochastic input-output AI** is a system that receives

- **Task Input** $x \in \Sigma^*$: User prompt, system prompt, user context, etc,

and (often)

- **Random sequence** $u \in \Sigma^*$ generated by using a pseudo random number generator until it satisfies the termination condition $u \in \mathcal{T}_x$,

and processes them by a program $p \in \Sigma^*$, then returns

Rough formulation of AI

We define an AI as an input-output stochastic computation process

Definition (A rough formulation of an AI)

A **stochastic input-output AI** is a system that receives

- **Task Input** $x \in \Sigma^*$: User prompt, system prompt, user context, etc,

and (often)

- **Random sequence** $u \in \Sigma^*$ generated by using a pseudo random number generator until it satisfies the termination condition $u \in \mathcal{T}_x$,

and processes them by a program $p \in \Sigma^*$, then returns

- **Output** $y = f_p(u, x) \in \Sigma^*$: response to the users, internal steps, etc.

where $f_p : \Sigma^* \rightarrow \Sigma^*$ is the function implemented by p .

Example 1: text-to-image using a diffusion process.

Example 1: text-to-image using a diffusion process.

Example (Text-to-image using diffusion process (e.g., [15]))

- **Input:** user prompt x
- **Step 1:** Convert the user prompt x to a context vector c (e.g., by CLIP).

Example 1: text-to-image using a diffusion process.

Example (Text-to-image using diffusion process (e.g., [15]))

- **Input:** user prompt x
- **Step 1:** Convert the user prompt x to a context vector c (e.g., by CLIP).
- **Step 2:** Generate an initial random latent image z_0 by generating pseudo random numbers.

Example 1: text-to-image using a diffusion process.

Example (Text-to-image using diffusion process (e.g., [15]))

- **Input:** user prompt x
- **Step 1:** Convert the user prompt x to a context vector c (e.g., by CLIP).
- **Step 2:** Generate an initial random latent image z_0 by generating pseudo random numbers.
- **Step 3:** For $t \leftarrow 0, 1, \dots, T - 1$: Compute next z_{t+1} from z_t, c, t .

Example 1: text-to-image using a diffusion process.

Example (Text-to-image using diffusion process (e.g., [15]))

- **Input:** user prompt x
- **Step 1:** Convert the user prompt x to a context vector c (e.g., by CLIP).
- **Step 2:** Generate an initial random latent image z_0 by generating pseudo random numbers.
- **Step 3:** For $t \leftarrow 0, 1, \dots, T - 1$: Compute next z_{t+1} from z_t, c, t .
- **Step 4:** From z_T , compute an image y (e.g., by VAE) and **output** y .

Example 1: text-to-image using a diffusion process.

Example (Text-to-image using diffusion process (e.g., [15]))

- **Input:** user prompt x
- **Step 1:** Convert the user prompt x to a context vector c (e.g., by CLIP).
- **Step 2:** Generate an initial random latent image z_0 by generating pseudo random numbers.
- **Step 3:** For $t \leftarrow 0, 1, \dots, T - 1$: Compute next z_{t+1} from z_t, c, t .
- **Step 4:** From z_T , compute an image y (e.g., by VAE) and **output** y .

It receives a user prompt x pseudo-random numbers and returns an image y , showing a pseudo-stochastic behavior. The termination random variable set $\mathcal{T}_x$ consists of fixed-length sequences.

Example 2: large language model

Example 2: large language model

Example (Large language model)

- **Input:** user prompt x

Example 2: large language model

Example (Large language model)

- **Input:** user prompt x
- **Step 1:** $t \leftarrow 0$, $y_{0:-1} = ()$.

Example 2: large language model

Example (Large language model)

- **Input:** user prompt x
- **Step 1:** $t \leftarrow 0$, $y_{0:-1} = ()$.
- **Step 2:** Compute the next token distribution Q_t from the user prompt x and past token sequence $y_{0:t-1}$.

Example 2: large language model

Example (Large language model)

- **Input:** user prompt x
- **Step 1:** $t \leftarrow 0$, $y_{0:-1} = ()$.
- **Step 2:** Compute the next token distribution Q_t from the user prompt x and past token sequence $y_{0:t-1}$.
- **Step 3:** From Q_t and a pseudo-random number, determine the next token y_t , and concatenate it to the past token sequence to get $y_{0:t} = y_{0:t-1} \cdot y_t$.

Example 2: large language model

Example (Large language model)

- **Input:** user prompt x
- **Step 1:** $t \leftarrow 0$, $y_{0:-1} = ()$.
- **Step 2:** Compute the next token distribution Q_t from the user prompt x and past token sequence $y_{0:t-1}$.
- **Step 3:** From Q_t and a pseudo-random number, determine the next token y_t , and concatenate it to the past token sequence to get $y_{0:t} = y_{0:t-1} \cdot y_t$.
- **Step 4:** If y_t is an EOS (end of sentence) token, then **output** $y_{0:t}$. Otherwise, go to **Step 2**.

Example 2: large language model

Example (Large language model)

- **Input:** user prompt x
- **Step 1:** $t \leftarrow 0$, $y_{0:-1} = ()$.
- **Step 2:** Compute the next token distribution Q_t from the user prompt x and past token sequence $y_{0:t-1}$.
- **Step 3:** From Q_t and a pseudo-random number, determine the next token y_t , and concatenate it to the past token sequence to get $y_{0:t} = y_{0:t-1} \cdot y_t$.
- **Step 4:** If y_t is an EOS (end of sentence) token, then **output** $y_{0:t}$. Otherwise, go to **Step 2**.

It also receives a user prompt x pseudo-random numbers and returns an image y , showing a pseudo-stochastic behavior. The termination random variable set $\mathcal{T}_x$ (all random number sequences that output EOS in the end) depends on x .

Behavior as a conditional probability mass function

Definition (AI-induced conditional probability mass function)

Let $\text{Prob}(\mathbf{u})$ be a probability mass virtually allocated to pseudo random number sequence $\mathbf{u}$.

For a stochastic input-output AI A with the termination condition given by $\mathcal{T}_x$, define Q_A by

$$Q_A(y \mid x) = \sum_{\mathbf{u} \in \mathcal{T}_x} \text{Prob}(\mathbf{u}) \mathbb{1}(f(\mathbf{u}, x) = y).$$

This is the conditional probability mass function representing the stochastic input-output AI.

Behavior as a conditional probability mass function

Definition (AI-induced conditional probability mass function)

Let $\text{Prob}(\mathbf{u})$ be a probability mass virtually allocated to pseudo random number sequence $\mathbf{u}$.

For a stochastic input-output AI A with the termination condition given by $\mathcal{T}_x$, define Q_A by

$$Q_A(y \mid x) = \sum_{\mathbf{u} \in \mathcal{T}_x} \text{Prob}(\mathbf{u}) \mathbb{1}(f(\mathbf{u}, x) = y).$$

This is the conditional probability mass function representing the stochastic input-output AI.

The goal of AI explanation: **to explain** Q_A .

Outline

1. Motivation

2. Problem Formulation

3. Formulating the 4 conditions

4. Main Result

5. Conclusion

6. References

Outline

1. Motivation

2. Problem Formulation

3. Formulating the 4 conditions

4. Main Result

5. Conclusion

6. References

Formulation of AI Performance: perplexity

As a metric of the AI performance, we simply use **perplexity**, widely used in NLP (e.g., [12]).

Definition (Perplexity)

Given an input $x \in \mathcal{X}$ and an output $y \in \mathcal{Y}$, the **perplexity** of a conditional probability mass function Q is

$$\frac{1}{Q(y \mid x)}.$$

Given a true distribution P on $\mathcal{X} \times \mathcal{Y}$, the expected logarithmic perplexity is

$$\mathbb{E}_{X,Y \sim P} \left[-\log Q(Y \mid X) \right].$$

Remark

A Small expected logarithmic perplexity means good AI performance.

Formulation of completely faithful explanations

A faithful explanation should recover the AI behavior when we interpret it under a fixed formal rule.

Formulation of completely faithful explanations

A faithful explanation should recover the AI behavior when we interpret it under a fixed formal rule.

We define an **interpretation function** to represent the rule to interpret the explanation.

Formulation of completely faithful explanations

A faithful explanation should recover the AI behavior when we interpret it under a fixed formal rule.

We define an **interpretation function** to represent the rule to interpret the explanation.

Definition (Interpretation function)

Fix a letter set Λ of natural languages (e.g., Unicode). An **interpretation function** is a **computable** function

$$\text{Interpret} : \subseteq \Lambda^* \times \mathbb{N} \times \mathcal{Z} \xrightarrow{\text{comp}} \mathbb{Q}$$

that interprets an explanation string as an arbitrary-precision approximation of a real-valued function.

Notes on a computable function

Informally speaking, a **computable function** is a function that can be realized by Python with unlimited time and memory. It includes, e.g., an LLM.

Notes on a computable function

Informally speaking, a **computable function** is a function that can be realized by Python with unlimited time and memory. It includes, e.g., an LLM.

In the above, we can replace Python with another programming language, e.g., C, because we can write a C emulator in Python and a Python emulator in C.

Notes on a computable function

Informally speaking, a **computable function** is a function that can be realized by Python with unlimited time and memory. It includes, e.g., an LLM.

In the above, we can replace Python with another programming language, e.g., C, because we can write a C emulator in Python and a Python emulator in C.

A function implemented in a programming language may not define the return value for some inputs owing to an infinite loop. Such a function is called a partial function, and is denoted like $f : \subseteq \mathcal{X} \rightarrow \mathcal{Y}$, instead of $f : \mathcal{X} \rightarrow \mathcal{Y}$.

When $f(x)$ is defined, we write $f(x) \downarrow$.

A formal definition of computable functions (Preliminary)

In the following, we define the set of string-input/output computable function.

We can convert the definition of computability to define the computability of functions with general source set $\mathcal{X}$ and target set $\mathcal{Y}$ by a fixed encoding rule, i.e., maps $\mathcal{X} \rightarrow \Sigma^*$ and $\mathcal{Y} \rightarrow \Sigma^*$, as long as $\mathcal{X}, \mathcal{Y}$ are at most countable.

A formal definition of computable functions (Part 1: primitive recursive)

Definition (Primitive recursive functions (string version))

Let the alphabet set Σ be a finite set such that $|\Sigma| \geq 2$ and let Σ^* denote the set of finite-length strings, i.e., $\Sigma^* := \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$. We define the set of primitive recursive functions according to the following rules.

- $\text{emp}, \text{append}_a : \Sigma^* \rightarrow \Sigma^*$ (for all $a \in \Sigma$) are primitive recursive functions, where
 - $\text{emp}(x) = ()$.
 - $\text{append}_a(x) = x \cdot a$ for all $a \in \Sigma$.
- If $f, g : \Sigma^* \rightarrow \Sigma^*$ are primitive recursive function, then $\text{comp}[f, g] : \Sigma^* \rightarrow \Sigma^*$ is also a primitive recursive function, where
 - $\text{comp}[f, g](x) = g(f(x))$.

A formal definition of computable functions (Part 1: primitive recursive)

Definition (Primitive recursive functions (string version) (continued))

Let the alphabet set Σ be a finite set such that $|\Sigma| \geq 2$ and let Σ^* denote the set of finite-length strings, i.e., $\Sigma^* := \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$. We define the set of primitive recursive functions according to the following rules.

- $(\text{emp}, \text{append}_a, \text{comp}[f, g])$ were primitive recursive functions)

A formal definition of computable functions (Part 1: primitive recursive)

Definition (Primitive recursive functions (string version) (continued))

Let the alphabet set Σ be a finite set such that $|\Sigma| \geq 2$ and let Σ^* denote the set of finite-length strings, i.e., $\Sigma^* := \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$. We define the set of primitive recursive functions according to the following rules.

- $(\text{emp}, \text{append}_a, \text{comp}[f, g])$ were primitive recursive functions)
- If $c, t_a : \Sigma^* \rightarrow \Sigma^*$ are a primitive recursive function for all $a \in \Sigma$, then $\text{iter}[c, (t_a)_{a \in \Sigma}]$ is also primitive recursive function, where
 - $\text{iter}[c, (t_a)_{a \in \Sigma}](x) = s_{|c|}$ where
 - $c = c(x)$
 - $s_k := \begin{cases} x & \text{if } k = 0, \\ t_{c_{k-1}}(s_{k-1}) & \text{if } k = 1, 2, \dots, |c|. \end{cases}$

A formal definition of computable functions (Part 1.5: partial functions)

In a computer program, we often define a loop that breaks under some condition. This may cause an infinite loop, so our definition must handle it.

A formal definition of computable functions (Part 1.5: partial functions)

In a computer program, we often define a loop that breaks under some condition. This may cause an infinite loop, so our definition must handle it. Idea: **(infinite loop) = (undefined)**.

A formal definition of computable functions (Part 1.5: partial functions)

In a computer program, we often define a loop that breaks under some condition. This may cause an infinite loop, so our definition must handle it. Idea: **(infinite loop) = (undefined)**.

Definition (Partial functions)

Let $\mathcal{X}$ and $\mathcal{Y}$ be sets. If $f(x) \in \mathcal{Y}$ is defined for a $x \in \mathcal{X}$, then we write $f(x) \downarrow$.

A formal definition of computable functions (Part 1.5: partial functions)

In a computer program, we often define a loop that breaks under some condition. This may cause an infinite loop, so our definition must handle it. Idea: **(infinite loop) = (undefined)**.

Definition (Partial functions)

Let $\mathcal{X}$ and $\mathcal{Y}$ be sets. If $f(x) \in \mathcal{Y}$ is defined for a $x \in \mathcal{X}$, then we write $f(x) \downarrow$.

We say that f is a **partial function** from the source set $\mathcal{X}$ to the target set $\mathcal{Y}$, and write $f : \subseteq \mathcal{X} \rightarrow \mathcal{Y}$.

A formal definition of computable functions (Part 1.5: partial functions)

In a computer program, we often define a loop that breaks under some condition. This may cause an infinite loop, so our definition must handle it. Idea: **(infinite loop) = (undefined)**.

Definition (Partial functions)

Let $\mathcal{X}$ and $\mathcal{Y}$ be sets. If $f(x) \in \mathcal{Y}$ is defined for a $x \in \mathcal{X}$, then we write $f(x) \downarrow$.

We say that f is a **partial function** from the source set $\mathcal{X}$ to the target set $\mathcal{Y}$, and write $f : \subseteq \mathcal{X} \rightarrow \mathcal{Y}$.

If $f(x)$ is not defined for a $x \in \mathcal{X}$, then we write $f(x) \uparrow$.

- In the computability theory context, $f(x) \uparrow$ indicates the program realizing f falls into an infinite loop.

A formal definition of computable functions (Part 2: partially computable)

Definition ((Partially) computable functions (string version))

We define the set of (partially) computable functions according to the following rules.

- $\text{emp}, \text{append}_a : \Sigma^* \rightarrow \Sigma^*$ are partially computable functions.
- If $f, g : \subseteq \Sigma^* \rightarrow \Sigma^*$ are partially computable function, then $\text{comp}[f, g] : \Sigma^* \rightarrow \Sigma^*$ is also partially computable, where
 - $\text{comp}[f, g](x) = g(f(x))$ if

A formal definition of computable functions (Part 2: partially computable)

Definition ((Partially) computable functions (string version))

We define the set of (partially) computable functions according to the following rules.

- $\text{emp}, \text{append}_a : \Sigma^* \rightarrow \Sigma^*$ are partially computable functions.
- If $f, g : \subseteq \Sigma^* \rightarrow \Sigma^*$ are partially computable function, then $\text{comp}[f, g] : \Sigma^* \rightarrow \Sigma^*$ is also partially computable, where
 - $\text{comp}[f, g](x) = g(f(x))$ if $f(x) \downarrow$ and $g(z) \downarrow$ where $z = f(x)$; otherwise undefined.

A formal definition of computable functions (Part 2: partially computable)

Definition ((Partially) computable functions (string version) (continued))

- $(\text{emp}, \text{append}_a, \text{comp}[f, g])$ were partially computable)
- If $c, t_a : \subseteq \Sigma^* \rightarrow \Sigma^*$ are partially computable for all $a \in \Sigma$, then $\text{iter}[c, (t_a)_{a \in \Sigma}]$ is also partially computable, where
 - $\text{iter}[c, (t_a)_{a \in \Sigma}](x) = s_{|c|}$ where
 - $c = c(x)$
 - $s_k := \begin{cases} x & \text{if } k = 0, \\ t_{c_{k-1}}(s_{k-1}) & \text{if } k = 1, 2, \dots, |c|. \end{cases}$

A formal definition of computable functions (Part 2: partially computable)

Definition ((Partially) computable functions (string version) (continued))

- $(\text{emp}, \text{append}_a, \text{comp}[f, g])$ were partially computable)
- If $c, t_a : \subseteq \Sigma^* \rightarrow \Sigma^*$ are partially computable for all $a \in \Sigma$, then $\text{iter}[c, (t_a)_{a \in \Sigma}]$ is also partially computable, where
 - $\text{iter}[c, (t_a)_{a \in \Sigma}](x) = s_{|c|}$ where
 - $c = c(x)$
 - $s_k := \begin{cases} x & \text{if } k = 0, \\ t_{c_{k-1}}(s_{k-1}) & \text{if } k = 1, 2, \dots, |c|. \end{cases}$
if $t_{c_{k-1}}(s_{k-1})$ is defined for $k = 1, 2, \dots, |c|$; otherwise undefined.

A formal definition of computable functions (Part 2: partially computable)

Definition ((Partially) computable functions (string version) (continued))

- $(\text{emp}, \text{append}_a, \text{comp}[f, g], \text{iter}[c, (t_a)_{a \in \Sigma}])$ were partially computable)
- If $p, t : \subseteq \Sigma^* \rightarrow \Sigma^*$ are partially computable for all $a \in \Sigma$, then $\text{while}[p, t]$ is also partially computable, where
- $\text{while}[p, t](x) = s_{k^*}$, where
 - $s_k := \begin{cases} x & \text{if } k = 0, \\ t(s_{k-1}) & \text{if } k > 0, \end{cases}$
 - $k^* := \min\{k \mid p(s_{k'}) \downarrow \text{ for all } k' = 0, 1, \dots, k \text{ and } p(s_k) = ()\}$.
if

A formal definition of computable functions (Part 2: partially computable)

Definition ((Partially) computable functions (string version) (continued))

- $(\text{emp}, \text{append}_a, \text{comp}[f, g], \text{iter}[c, (t_a)_{a \in \Sigma}])$ were partially computable)
- If $p, t : \subseteq \Sigma^* \rightarrow \Sigma^*$ are partially computable for all $a \in \Sigma$, then $\text{while}[p, t]$ is also partially computable, where
- $\text{while}[p, t](x) = s_{k^*}$, where
 - $s_k := \begin{cases} x & \text{if } k = 0, \\ t(s_{k-1}) & \text{if } k > 0, \end{cases}$
 - $k^* := \min\{k \mid p(s_{k'}) \downarrow \text{ for all } k' = 0, 1, \dots, k \text{ and } p(s_k) = ()\}$.
if $\min\{k \mid p(s_{k'}) \downarrow \text{ for all } k' = 0, 1, \dots, k \text{ and } p(s_k) = ()\}$ is non-empty; otherwise undefined.

An infinite loop caused by "while"

The following pseudocode of $\text{while}[p, t]$ shows why an infinite loop may occur.

- **Input:** x .
- **Initialize** $s_0 := x$.
- **For** $k = 0, 1, 2, \dots$:
 - **Compute** $s_k := t(s_{k-1})$.
 - **If** $p(s_k) = ()$, then **return** s_k ; **otherwise continue**.

Possibly, $p(s_k) = ()$ is never satisfied; then it leads to an infinite loop.

Faithfulness as reproducibility under a fixed interpretation rule

Definition (Completely faithful explanation)

A string $e \in \Lambda^*$ is a completely faithful explanation of a function

$$f : \mathcal{Z} \xrightarrow{\text{comp}} \mathbb{R}$$

under a fixed computable function $\text{Interpret} : \Lambda^* \times \mathbb{N} \times \mathcal{Z} \rightarrow \mathbb{Q}$ if, for every $z \in \mathcal{Z}$ and every $k \in \mathbb{N}$,

$$|\text{Interpret}(e, k, z) - f(z)| < |\Sigma|^{-k}.$$

Faithfulness as reproducibility under a fixed interpretation rule

Definition (Completely faithful explanation)

A string $e \in \Lambda^*$ is a completely faithful explanation of a function

$$f : \mathcal{Z} \xrightarrow{\text{comp}} \mathbb{R}$$

under a fixed computable function $\text{Interpret} : \Lambda^* \times \mathbb{N} \times \mathcal{Z} \rightarrow \mathbb{Q}$ if, for every $z \in \mathcal{Z}$ and every $k \in \mathbb{N}$,

$$|\text{Interpret}(e, k, z) - f(z)| < |\Sigma|^{-k}.$$

Remark

Considering the approximation is necessary since a computer cannot directly handle irrational numbers.

Formulating the length of an explanation

The length of the explanation is $|e|$, the length as a sequence.

$$|A \text{ good } AI| = 9.$$

Key idea for formulating the operating environment complexity

How do we quantify the complexity of the input-output relation?

Key idea for formulating the operating environment complexity

How do we quantify the complexity of the input-output relation?

If the relation is simple, we can find a simple rule that generates the output from the given input.

Key idea for formulating the operating environment complexity

How do we quantify the complexity of the input-output relation?

If the relation is simple, we can find a simple rule that generates the output from the given input.

Our idea: Use **the shortest program length** generating the output from the given input as a measure of the input-output relation complexity.

Key idea for formulating the operating environment complexity

How do we quantify the complexity of the input-output relation?

If the relation is simple, we can find a simple rule that generates the output from the given input.

Our idea: Use **the shortest program length** generating the output from the given input as a measure of the input-output relation complexity.

This idea leads us to the definition of **conditional (plain) Kolmogorov complexity**.

Preliminary to define the conditional Kolmogorov complexity

To formally handle multiple variables, we need to a delimiter. For example,

Preliminary to define the conditional Kolmogorov complexity

To formally handle multiple variables, we need to a delimiter. For example, if you concatenate 000 and 0 directly, then we cannot distinguish it from the concatenation of 00 and 00.

Preliminary to define the conditional Kolmogorov complexity

To formally handle multiple variables, we need to a delimiter. For example, if you concatenate 000 and 0 directly, then we cannot distinguish it from the concatenation of 00 and 00.

For formal delimitation, We introduce self-delimiting encoding function.

Definition (Prefix-free set and self-delimiting encoding function)

A set of strings $\mathcal{F} \subseteq \Sigma^*$ is **prefix-free** if, for every $c \in \mathcal{F}$ and every $z \in \Sigma^* \setminus \{()\}$, $c \cdot z \notin \mathcal{F}$.

Preliminary to define the conditional Kolmogorov complexity

To formally handle multiple variables, we need to a delimiter. For example, if you concatenate 000 and 0 directly, then we cannot distinguish it from the concatenation of 00 and 00.

For formal delimitation, We introduce self-delimiting encoding function.

Definition (Prefix-free set and self-delimiting encoding function)

A set of strings $\mathcal{F} \subseteq \Sigma^*$ is **prefix-free** if, for every $c \in \mathcal{F}$ and every $z \in \Sigma^* \setminus \{()\}$, $c \cdot z \notin \mathcal{F}$.

A total computable function $\bar{\bullet} : \Sigma^* \xrightarrow{\text{comp}} \Sigma^*$, where $\bullet$ is a placeholder, is a **self-delimiting encoding function** if $\bar{\bullet}$ is an injective computable function whose image is a computable, or decidable, prefix-free set.

Preliminary to define the conditional Kolmogorov complexity

Definition (Universal conditional function)

A partial computable function $U : \subseteq \Sigma^* \xrightarrow{\text{comp}} \Sigma^*$ is a **universal conditional (partial computable) function** if there exists a decidable prefix-free set $\text{Programs}_U \subseteq \Sigma^*$ such that, for every partial computable function $f : \subseteq \Sigma^* \xrightarrow{\text{comp}} \Sigma^*$, there exists a string $p \in \text{Programs}_U$ for which the equality of partial functions $U(\bar{\bullet} \cdot p \cdot \diamond) = f(\bar{\bullet} \cdot \diamond)$ holds, including their domains.

Preliminary to define the conditional Kolmogorov complexity

Definition (Universal conditional function)

A partial computable function $U : \subseteq \Sigma^* \xrightarrow{\text{comp}} \Sigma^*$ is a **universal conditional (partial computable) function** if there exists a decidable prefix-free set $\text{Programs}_U \subseteq \Sigma^*$ such that, for every partial computable function $f : \subseteq \Sigma^* \xrightarrow{\text{comp}} \Sigma^*$, there exists a string $p \in \text{Programs}_U$ for which the equality of partial functions $U(\bar{\bullet} \cdot p \cdot \diamond) = f(\bar{\bullet} \cdot \diamond)$ holds, including their domains.

More concretely, $U(\bar{\bullet} \cdot p \cdot \diamond) = f(\bar{\bullet} \cdot \diamond)$ means that, for every $x, w \in \Sigma^*$, the following holds:

$$\begin{cases} U(\bar{x} \cdot p \cdot w) \downarrow = f(\bar{x} \cdot w) \downarrow & \text{if } \bar{x} \cdot w \in \text{dom } f, \\ U(\bar{x} \cdot p \cdot w) \uparrow & \text{if } \bar{x} \cdot w \notin \text{dom } f. \end{cases}$$

Preliminary to define the conditional Kolmogorov complexity

Definition (Universal conditional function)

A partial computable function $U : \subseteq \Sigma^* \xrightarrow{\text{comp}} \Sigma^*$ is a **universal conditional (partial computable) function** if there exists a decidable prefix-free set $\text{Programs}_U \subseteq \Sigma^*$ such that, for every partial computable function $f : \subseteq \Sigma^* \xrightarrow{\text{comp}} \Sigma^*$, there exists a string $p \in \text{Programs}_U$ for which the equality of partial functions $U(\bar{\bullet} \cdot p \cdot \diamond) = f(\bar{\bullet} \cdot \diamond)$ holds, including their domains.

- A programming language interpreter (w/ some processing) is an example of U
- p can be regarded as a program interpreted by U implementing the function f .

Formulation of the operating environment complexity

Definition (Conditional plain Kolmogorov complexity)

Fix a **universal conditional function** U (like a Python interpreter). For $x, y \in \Sigma^*$, the **conditional plain Kolmogorov complexity** is defined by

$$C_U(y \mid x) := \min \{ |p| \mid U(\bar{x} \cdot p) \downarrow = y \}.$$

$\bar{\bullet}$ is a fixed self-delimiting encoder function.

Formulation of the operating environment complexity

Definition (Conditional plain Kolmogorov complexity)

Fix a **universal conditional function** U (like a Python interpreter). For $x, y \in \Sigma^*$, the **conditional plain Kolmogorov complexity** is defined by

$$C_U(y \mid x) := \min \{ |p| \mid U(\bar{x} \cdot p) \downarrow = y \}.$$

$\bar{\cdot}$ is a fixed self-delimiting encoder function.

In short, $C_U(y \mid x)$ is the **shortest program length** to output y from x .

Formulation of the operating environment complexity

Definition (Conditional plain Kolmogorov complexity)

Fix a **universal conditional function** U (like a Python interpreter). For $x, y \in \Sigma^*$, the **conditional plain Kolmogorov complexity** is defined by

$$C_U(y \mid x) := \min \{ |p| \mid U(\bar{x} \cdot p) \downarrow = y \}.$$

$\bar{\cdot}$ is a fixed self-delimiting encoder function.

In short, $C_U(y \mid x)$ is the **shortest program length** to output y from x .

We use $\mathbb{E}_{X,Y} C_U(Y \mid X)$ as a metric of the operating environment complexity.

Formulation of the operating environment complexity

Definition (Conditional plain Kolmogorov complexity)

Fix a **universal conditional function** U (like a Python interpreter). For $x, y \in \Sigma^*$, the **conditional plain Kolmogorov complexity** is defined by

$$C_U(y \mid x) := \min \{ |p| \mid U(\bar{x} \cdot p) \downarrow = y \}.$$

$\bar{\cdot}$ is a fixed self-delimiting encoder function.

In short, $C_U(y \mid x)$ is the **shortest program length** to output y from x .

We use $\mathbb{E}_{X,Y} C_U(Y \mid X)$ as a metric of the operating environment complexity.

Note: changing U only changes an additive constant independent of x and y .

Outline

1. Motivation

2. Problem Formulation

3. Formulating the 4 conditions

4. Main Result

5. Conclusion

6. References

Outline

- 1. Motivation
- 2. Problem Formulation
- 3. Formulating the 4 conditions

4. Main Result

- 5. Conclusion
- 6. References

Fundamental Quadrilemma: a rough formula

Our theorem roughly states that:

Fundamental Quadrilemma: a rough formula

Our theorem roughly states that:

Up to logarithmic terms,

$$(\text{perplexity}) + (\text{faithful explanation length}) \geq (\text{conditional Kolmogorov complexity})$$

Fundamental Quadrilemma: a rough formula

Our theorem roughly states that:

Up to logarithmic terms,

$$(\text{perplexity}) + (\text{faithful explanation length}) \geq (\text{conditional Kolmogorov complexity})$$

Implication: If the operating environment (task) is complex, then the RHS is large.

Then, either of the following holds:

Fundamental Quadrilemma: a rough formula

Our theorem roughly states that:

Up to logarithmic terms,

$$(\text{perplexity}) + (\text{faithful explanation length}) \geq (\text{conditional Kolmogorov complexity})$$

Implication: If the operating environment (task) is complex, then the RHS is large.

Then, either of the following holds:

- The perplexity is large (poor performance), or

Fundamental Quadrilemma: a rough formula

Our theorem roughly states that:

Up to logarithmic terms,

$$(\text{perplexity}) + (\text{faithful explanation length}) \geq (\text{conditional Kolmogorov complexity})$$

Implication: If the operating environment (task) is complex, then the RHS is large.

Then, either of the following holds:

- The perplexity is large (poor performance), or
- The (faithful) explanation is long (uninterpretable explanation).

Fundamental Quadrilemma: mathematical form

Theorem (Fundamental Quadrilemma)

Fix a universal function U , an encoding function an alphabet Σ , an interpretation function with character set Λ , and $L_\Lambda := \max_{\lambda \in \Lambda} |\text{Enc}_\Lambda^{\text{PF}}(\lambda)|$.

Suppose that for every $x \in \mathcal{X}$, a completely faithful explanation e_x of $Q_A(\cdot | x)$ is given. For every probability distribution P on $\mathcal{X} \times \mathcal{Y}$,

$$\begin{aligned} \mathbb{E}_{X,Y \sim P} [-\log Q_A(Y | X)] + \mathbb{E}_{X,Y \sim P} [L_\Lambda |e_X| + 2 \log (L_\Lambda |e_X| + 1)] \\ \geq \mathbb{E}_{X,Y \sim P} C_U(Y | X) + c. \end{aligned}$$

The constant c does not depend on A , x , y , or P .

If Λ is Unicode characters and UTF-8 for encoding, $L_\Lambda = 32$.

Proof sketch

- It is sufficient to prove
$$-\log Q_A(\mathbf{y} \mid \mathbf{x}) + L_\Lambda |e_x| + 2 \log (L_\Lambda |e_x| + 1) \geq C_U(\mathbf{y} \mid \mathbf{x}) + c..$$
- We can construct a computable function that output $\mathbf{y}$ with the description of $Q_A(\cdot \mid \mathbf{x})$ and the code whose length is roughly $-\log Q_A(\mathbf{y} \mid \mathbf{x})$.
- From the definition of the interpretation function, $L_\Lambda |e_x| + 2 \log (L_\Lambda |e_x| + 1)$ is sufficient to describe $Q_A(\cdot \mid \mathbf{x})$.
- Hence, we can construct a programme that outputs $\mathbf{y}$ from $\mathbf{x}$ and the length is roughly $-\log Q_A(\mathbf{y} \mid \mathbf{x}) + L_\Lambda |e_x| + 2 \log (L_\Lambda |e_x| + 1)$.

Meaning of the inequality

Remark (Trade-off)

The theorem implies that the following cannot all be small or favorable:

1. performance (expected logarithmic perplexity): $\mathbb{E}[-\log Q_A(Y | X)]$,
2. expected faithful explanation length, $\mathbb{E}[|e_X|]$,
3. environment complexity, $\mathbb{E}C_U(Y | X)$.

Remark

Recall that a lengthy explanation is NOT interpretable.

Implication for AI governance

Remark (Governance implication)

In many practical applications:

1. the operation environment is complex,
2. good AI performance is required,
3. interpretable explanations are required.

Therefore, the condition to give up is usually complete faithfulness.

Remark

AI governance should be designed on the premise that AI explanations are incomplete in faithfulness.

Local and global explanations

Remark

The theorem is stated for local explanations e_x of $Q_A(\cdot \mid x)$. A global explanation of $Q_A(\cdot \mid \cdot)$ must be able to recover each local behavior $Q_A(\cdot \mid x)$. Hence, up to fixed constants, the lower-bound logic also applies to global explanations.

Hypothesis on the environment complexity.

Since the environment itself is complex, computing its complexity (e.g., $\mathbb{E}C_U(Y \mid X)$) is, in principle, a difficult problem.

Hypothesis on the environment complexity.

Since the environment itself is complex, computing its complexity (e.g., $\mathbb{E}C_U(Y | X)$) is, in principle, a difficult problem.

Nevertheless, our quadrilemma theorem is suggestive. If an AI that achieves the true input-output relation and is written by a short program were realized, then $\mathbb{E}_{X,Y \sim P}C_U(Y | X)$ would be small, and the quadrilemma would not be a problem. However, considering the history in which text generation could not achieve sufficient performance until the advent of large-scale language models, one can formulate the hypothesis that such a situation will not occur.

Outline

1. Motivation
2. Problem Formulation
3. Formulating the 4 conditions

4. Main Result

5. Conclusion
6. References

Outline

1. Motivation
2. Problem Formulation
3. Formulating the 4 conditions
4. Main Result
- 5. Conclusion**
6. References

Summary

Since shortness is a necessary condition for human interpretability, the theorem gives a mathematical quadrilemma among:

1. environment complexity,
2. AI performance,
3. interpretability,
4. complete faithfulness.

Summary

Since shortness is a necessary condition for human interpretability, the theorem gives a mathematical quadrilemma among:

1. environment complexity,
2. AI performance,
3. interpretability,
4. complete faithfulness.

AI governance should be designed on the premise that AI explanations are incomplete in faithfulness.

Outline

1. Motivation
2. Problem Formulation
3. Formulating the 4 conditions
4. Main Result
- 5. Conclusion**
6. References

Outline

1. Motivation
2. Problem Formulation
3. Formulating the 4 conditions
4. Main Result
5. Conclusion

6. References

References i

- [1] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Bannetot, Siham Tabik, Alberto Barbado, Salvador García, Sergio Gil-López, Daniel Molina, Richard Benjamins, et al. Explainable artificial intelligence (xai): Concepts, taxonomies, opportunities and challenges toward responsible ai. Information fusion, 58:82–115, 2020.
- [2] Marc Brysbaert. How many words do we read per minute? a review and meta-analysis of reading rate. Journal of memory and language, 109: 104047, 2019.

References ii

- [3] Guinness World Records. Most decimal places of pi memorized, 2015. URL <https://www.guinnessworldrecords.com/world-records/most-pi-places-memorised>. Record achieved by Rajveer Meena at VIT University, Vellore, India, on 21 March 2015.
- [4] Jan Hatzius, Joseph Briggs, and Devesh Kodnani. The potentially large effects of artificial intelligence on economic growth. Technical report, Goldman Sachs Global Investment Research, 2023.
- [5] Shiyuan Huang, Siddarth Mamidanna, Shreedhar Jangam, Yilun Zhou, and Leilani H Gilpin. Can large language models explain themselves? a study of llm-generated self-explanations. [arXiv preprint arXiv:2310.11207](https://arxiv.org/abs/2310.11207), 2023.

References iii

- [6] Yeo Wei Jie, Ranjan Satapathy, Rick Goh, and Erik Cambria. How interpretable are reasoning explanations from prompting large language models? In Findings of the Association for Computational Linguistics: NAACL 2024, pages 2148–2164, 2024.
- [7] Nicholas Kroeger, Dan Ley, Satyapriya Krishna, Chirag Agarwal, and Himabindu Lakkaraju. In-context explainers: Harnessing llms for explaining black box models. arXiv preprint arXiv:2310.05797, 2023.
- [8] Prabha M Kumarage and Mirka Saarela. Explainable generative ai: A two-stage review of existing techniques and future research directions. AI, 7 (1):31, 2026.

References iv

- [9] Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. Advances in neural information processing systems, 30, 2017.
- [10] McKinsey Global Institute. The economic potential of generative ai: The next productivity frontier, 2023.
- [11] National Institute of Standards and Technology. Artificial intelligence risk management framework: Generative artificial intelligence profile. Technical Report NIST AI 600-1, National Institute of Standards and Technology, 2024. URL <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>.

References v

- [12] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. OpenAI Blog, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.
- [13] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. " why should i trust you?" explaining the predictions of any classifier. In Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, pages 1135–1144, 2016.
- [14] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. Anchors: High-precision model-agnostic explanations. In Proceedings of the AAAI conference on artificial intelligence, volume 32, 2018.

References vi

- [15] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 10684–10695, 2022.
- [16] Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In Proceedings of the IEEE international conference on computer vision, pages 618–626, 2017.
- [17] Mattia Setzu, Riccardo Guidotti, Anna Monreale, Franco Turini, Dino Pedreschi, and Fosca Giannotti. Glocalx-from local to global explanations of black box ai models. Artificial Intelligence, 294:103457, 2021.

References vii

- [18] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In International conference on machine learning, pages 3319–3328. PMLR, 2017.
- [19] Giorgio Visani, Vincenzo Stanzione, and Damien Garreau. Gleams: Bridging the gap between local and global explanations. arXiv preprint arXiv:2408.05060, 2024.

Outline

7. Appendix

8. Why Not Use the Naive Complexity?

Outline

7. Appendix

8. Why Not Use the Naive Complexity?

Stochastic input-output AI

Let $\mathcal{X}$ be the input space and $\mathcal{Y}$ be the output space.

Definition (Stochastic input-output AI)

A stochastic input-output AI with domain $\mathcal{X}' \subseteq \mathcal{X}$ is a pair

$$A = (f, \tau),$$

where

$$f : \subseteq (\mathbb{N}_{<n})^* \times \mathcal{X} \xrightarrow{\text{comp}} \mathcal{Y}$$

is the main function, and

$$\tau : \subseteq (\mathbb{N}_{<n})^* \times \mathcal{X} \xrightarrow{\text{comp}} \{0, 1\}$$

is the random-sequence acceptance decision function.

Accepted random sequences

For each input $x \in \mathcal{X}'$, define

$$\mathcal{T}_x := \{\mathbf{u} \in (\mathbb{N}_{<n})^* \mid \tau(\mathbf{u}, x) \downarrow = 1\}.$$

Definition (Key requirements)

For each $x \in \mathcal{X}'$:

1. $\mathcal{T}_x$ is computable,
2. $\mathcal{T}_x$ is prefix-free,
3. a computable stopping bound $N_{\text{stop}}(x, k)$ satisfies

$$1 - \sum_{\substack{\mathbf{u} \in \mathcal{T}_x \\ |\mathbf{u}| \leq N_{\text{stop}}(x, k)}} n^{-|\mathbf{u}|} \leq |\Sigma|^{-k},$$

4. if $\mathbf{u} \in \mathcal{T}_x$, then $(\mathbf{u}, x) \in \text{dom } f$.

Intended stochastic operation

Definition (Intended operation)

Given $x \in \mathcal{X}'$, the AI repeatedly samples symbols from a uniform oracle $\text{Uniform}_{<n}$ and concatenates them into a random string u .

When $u \in \mathcal{T}_x$, it outputs

$$f(u, x)$$

and terminates.

Remark

This framework includes deterministic functions, diffusion models with fixed random-number consumption, and LLMs whose number of consumed random numbers depends on generated tokens and termination conditions.

Outline

7. Appendix

8. Why Not Use the Naive Complexity?

Outline

7. Appendix

8. Why Not Use the Naive Complexity?

Naive idea

A natural first attempt is to measure environment complexity by the Kolmogorov complexity of the true conditional probability mass function:

$$C_U(P(\cdot \mid \cdot)).$$

Proposition (Naive exact-minimization result)

If Q has a completely faithful explanation e with

$$L_\Lambda|e| < C_U(P) - c_{\text{Interpret}},$$

then Q cannot exactly minimize expected logarithmic conditional perplexity.

Remark

This only rules out exact minimization.

Failure of the naive method

Proposition (Failure of the naive lower bound)

Let $\mathcal{X}$ and $\mathcal{Y}$ be countably infinite. For every $n \in \mathbb{N}$ and every $\epsilon > 0$, there exist π , computable P, Q , and a completely faithful explanation e of Q such that

$$\mathbb{E}_{X \sim \pi, Y | X \sim P}[-\log Q(Y | X)] - \min_{Q^*} \mathbb{E}_{X \sim \pi, Y | X \sim P}[-\log Q^*(Y | X)] < \epsilon,$$

but

$$|e| < C_U(P) - n.$$

Remark

Thus $C_U(P)$ does not give the desired performance-explanation trade-off near the optimum.

Why expected conditional Kolmogorov complexity is used

Remark (Chosen complexity indicator)

The theorem uses

$$\mathbb{E}_{X,Y \sim P} C_U(Y \mid X)$$

because it measures the algorithmic complexity of individual true outputs conditioned on inputs.

Remark

Replacing it by entropy alone loses the implication for explanation length, because Gibbs' inequality already gives

$$\mathbb{E}_{X,Y \sim P} [-\log Q(Y \mid X)] \geq H_P(Y \mid X).$$

Gap from entropy

Proposition (Kolmogorov complexity can exceed entropy)

For every $n \in \mathbb{N}$, there exists a probability distribution P such that

$$\mathbb{E}_{X,Y \sim P} K_V(Y \mid X) \geq H_P(Y \mid X) + n.$$

Remark

This shows that the main theorem can yield a stronger lower bound than an entropy-only argument.